

Computer Vision

Exploring Feature Detection: A Comparative Study of Classical and Deep Learning Methods Across Complex Scenes

René Mourambou Diyema

`rene.mourambou-diyema@universite-paris-saclay.fr`
`garciagarcia293@gmail.com`

Université Paris-Saclay

Contents

1	Introduction	3
2	State of the Art	4
2.1	Classical Methods : SIFT and SURF	4
2.2	Deep Learning Methods : SuperPoints and D2-Net	4
2.3	Challenges in Feature Detection	4
3	Methodology	5
3.1	Datasets	5
3.1.1	Dataset Construction	5
3.1.2	Image Preprocessing	5
3.2	Implementation	5
3.2.1	Detector Configuration	5
3.2.2	Feature Matching Algorithm	6
3.2.3	Software Architecture	6
3.3	Evaluation Metrics	6
3.3.1	Number of Features Detected	6
3.3.2	Number of Matches	7
3.3.3	Matching Efficiency	7
3.3.4	Average Matching Distance	7
3.3.5	Spatial Distribution Entropy	7
4	Experimental Results	8
4.1	Repetitive Patterns	8
4.2	Cluttered Backgrounds	9
4.3	High Illumination	11
4.4	Cross-Dataset Synthesis	12
5	Discussion	13
5.1	Comparison with Zhou (2024)	13
5.2	Critical Analysis	13
6	Conclusion	14
A	Performance Ranking Summary	15
A.1	Best Performers by Metric and Scenario	15
B	Algorithm Pseudocode	15
B.1	SIFT Detection Algorithm	15
B.2	SuperPoint Network Architecture	16
B.3	Brute-Force Matching with Cross-Check	16
C	Experimental Configuration Details	17
C.1	Hardware and Software Environment	17
C.2	Complete Parameter Configuration	17

1 Introduction

Feature detection and matching constitute fundamental operations in computer vision, enabling applications ranging from image stitching and 3D reconstruction to object recognition and visual odometry. The extraction of robust, discriminative keypoints that remain invariant to geometric and photometric transformations has been a central challenge in the field for decades. Traditional hand-crafted approaches, exemplified by (Scale-Invariant Feature Transform) and (Speeded-Up Robust Features), dominated this domain for over two decades through careful engineering of multi-scale representations and gradient-based descriptors. These methods achieved remarkable success by exploiting principles from differential geometry and signal processing theory.

The emergence of deep learning has fundamentally transformed feature detection methodologies. Modern convolutional neural networks can learn hierarchical representations directly from data, potentially capturing patterns that elude hand-crafted designs. Methods such as and leverage self-supervised learning and joint detection-description architectures to achieve superior performance in challenging scenarios. This paradigm shift raises important questions about the relative merits of classical versus learned approaches, particularly regarding their generalization capabilities, computational requirements, and practical applicability.

This project aims to conduct a rigorous empirical comparison between classical and deep learning-based feature detection methods by replicating the experimental framework proposed by Zhou (2024). Our specific objectives include: (1) implementing and evaluating , , and on standardized image datasets, (2) quantifying their performance across three distinct challenging scenarios using multiple evaluation metrics, (3) conducting statistical significance testing to validate observed performance differences, and (4) analyzing the practical trade-offs between computational efficiency, detection quality, and robustness to environmental variations. Through this comprehensive analysis, we seek to provide evidence-based guidance for algorithm selection in real-world computer vision applications.

2 State of the Art

2.1 Classical Methods : SIFT and SURF

SIFT (Scale-Invariant Feature Transform) detects and describes local features robust to scale, rotation, illumination, and viewpoint changes through four main stages. First, a scale-space representation is constructed: $L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$ and extrema are detected using the Difference-of-Gaussian: $D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$. Keypoints are refined via contrast and edge filtering, assigned consistent orientations using gradient histograms, and described with 128-dimensional histograms over 4×4 subregions.

SURF (Speeded-Up Robust Features) accelerates detection using integral images: $I_\Sigma(x, y) = \sum_{i \leq x} \sum_{j \leq y} I(i, j)$ and approximates the Hessian determinant with box filters: $\det(\mathcal{H}_{\text{approx}}) = D_{xx}D_{yy} - (0.9D_{xy})^2$. Keypoints are detected at maxima of this response, and descriptors are computed via Haar wavelet responses in circular regions, producing 64- or 128-dimensional vectors.

2.2 Deep Learning Methods : SuperPoints and D2-Net

SuperPoint uses a shared encoder with two decoder heads for detection and description. The detector outputs a tensor of size $H_c \times W_c \times 65$ and selects keypoints with softmax and non-maximum suppression, while the descriptor head generates 256-dimensional L2-normalized descriptors. Homographic Adaptation trains the network to detect keypoints consistently across random homographies, enforcing descriptor similarity for corresponding points and dissimilarity for non-corresponding points.

D2-Net jointly detects and describes features using a VGG16 encoder. The channel-wise and spatial scores are computed as:

$$\alpha_{ij}^c = \frac{\exp(F_{ij}^c/\tau)}{\sum_{c'} \exp(F_{ij}^{c'}/\tau)}, \quad \beta_{ij}^c = \begin{cases} 1 & \text{if } F_{ij}^c = \max_{(i', j') \in \mathcal{N}(i, j)} F_{i'j'}^c \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

The overall detection score is $\gamma_{ij} = \max_c(\alpha_{ij}^c \cdot \beta_{ij}^c)$, and high- γ points serve as keypoints with L2-normalized descriptors.

2.3 Challenges in Feature Detection

Feature detection is challenged by repetitive patterns, cluttered backgrounds, and illumination variations. Repetitive structures create descriptor ambiguity, while cluttered scenes produce numerous low-distinctiveness keypoints. Illumination changes alter local intensities and gradients, requiring classical methods to rely on gradient orientations:

$$\text{orientation} = \arctan \frac{L_y}{L_x} \quad (2)$$

and learned methods to train on diverse lighting conditions. Robust algorithms must balance detection quality, distinctiveness, and invariance under these challenges.

3 Methodology

3.1 Datasets

3.1.1 Dataset Construction

We constructed three specialized image datasets to evaluate feature detection performance under distinct challenging conditions. Each dataset comprises 25 image pairs (50 individual images), yielding 75 total pairs across all scenarios. The image pairs were carefully selected to represent transformations commonly encountered in real-world applications, including geometric distortions (rotation, scaling, translation), photometric variations, and scene complexity changes. Images were sourced to match the characteristics described in the original study by Zhou (2024), focusing on the three specific challenge categories.

The *Repeating Patterns* dataset contains images featuring self-similar structures such as architectural facades, tiled surfaces, and periodic textures. These scenes test the algorithms’ ability to establish correct correspondences despite the presence of visually identical or nearly identical regions at multiple spatial locations. The *Cluttered Backgrounds* dataset presents complex scenes with dense arrangements of objects, overlapping elements, and rich textural content. These images challenge feature detectors to identify salient keypoints amidst abundant visual information and potential occlusions. The *High Lighting* dataset includes images captured under strong directional illumination, creating pronounced shadows, highlights, and significant local intensity variations that test photometric robustness.

3.1.2 Image Preprocessing

All images underwent standardized preprocessing to ensure consistent evaluation conditions. Each image was resized to a uniform resolution of 512×512 pixels using bilinear interpolation, maintaining aspect ratio through center cropping or padding as necessary. Color images were converted to grayscale using standard luminance weightings to eliminate chromatic information and focus evaluation on structural features. No additional preprocessing steps such as histogram equalization or noise reduction were applied, allowing algorithms to demonstrate their inherent robustness to natural image characteristics.

3.2 Implementation

3.2.1 Detector Configuration

We configured each feature detection algorithm according to established best practices and parameter values commonly employed in the literature. For SIFT, we set the contrast threshold to 0.04 to filter low-contrast keypoints, the edge threshold to 10 to eliminate poorly localized edge responses, and retained the default values for the number of octave layers (3) and initial Gaussian blur sigma (1.6). These parameters align with Lowe’s original recommendations and provide a reasonable balance between detection quantity and quality.

For SuperPoint, we configured the non-maximum suppression distance to 4 pixels and set the confidence threshold to 0.015 for keypoint detection. These values were selected based on the original paper’s recommendations for balanced performance. The network operated in CPU mode rather than GPU acceleration due to hardware constraints, with inference performed using the pretrained model weights from the original authors. All

processing was conducted on a system equipped with an Intel processor and 16GB RAM, with computation performed entirely on CPU to ensure consistent experimental conditions.

For D2-Net, we employed the VGG16-based architecture with ReLU activation on the final feature map. Maximum image edge dimensions were set to 1600 pixels, with a maximum sum of edges of 2800 pixels to control memory consumption. The model operated without multiscale processing to maintain computational feasibility. As with SuperPoint, D2-Net processing occurred on CPU using pretrained model weights.

We attempted to include SURF in our experimental comparison, as specified in the original study by Zhou (2024). However, SURF is subject to patent restrictions and is not included in standard distributions of OpenCV. The algorithm’s implementation requires enabling non-free algorithms through specific compilation flags, which were not available in our installation environment (OPENCV_ENABLE_NONFREE). Multiple attempts to install OpenCV with SURF support through various package management systems (pip versions 4.5.3–4.5.5) failed due to these licensing restrictions. Given the time constraints and the primary focus on comparing classical hand-crafted methods against deep learning approaches, we proceeded with the three successfully implemented algorithms: SIFT (representing classical methods), SuperPoint and D2-Net (representing deep learning methods). This modification does not compromise the core objectives of our study, as SIFT provides a robust representation of traditional feature detection methodologies.

3.2.2 Feature Matching Algorithm

Feature matching employed the Brute-Force Matcher with L2 norm distance metric, consistently applied across all detectors. The matcher operates by computing pairwise distances between all descriptors from the first and second images, selecting the minimum-distance pairing for each query descriptor. We enabled cross-checking to ensure mutual nearest-neighbor relationships, wherein a match between descriptors d_1 from image 1 and d_2 from image 2 is retained only if d_2 is also the nearest neighbor of d_1 . This bidirectional verification significantly reduces false matches at the cost of lower match counts. No ratio test was applied to maintain uniform evaluation conditions across all methods.

3.2.3 Software Architecture

The implementation follows a modular architecture with distinct components for image loading, feature detection, matching, evaluation, and visualization. The codebase is organized into five primary modules: `config` containing experimental parameters, `models` implementing detector interfaces, `utils` providing image processing and matching utilities, `visualization` generating plots and figures, and `experiments` orchestrating the experimental pipeline. This design promotes code reusability and facilitates systematic experimentation. All experiments were executed through a Jupyter notebook interface that sequentially invokes the experimental runner for each dataset, with automatic result persistence to disk.

3.3 Evaluation Metrics

3.3.1 Number of Features Detected

The raw count of detected keypoints in each image provides a basic measure of detection density. This metric is computed independently for each image in a pair, yielding two

values (N_1, N_2) that characterize the detector’s response to specific scene content. While higher feature counts might suggest richer scene representation, they do not necessarily correlate with matching quality if many features lack distinctiveness.

3.3.2 Number of Matches

The total count of successfully matched keypoint pairs N_m between two images directly indicates the algorithm’s ability to establish correspondences. This metric depends both on detection quality (are distinctive regions detected?) and descriptor discriminability (can corresponding regions be reliably matched?). Higher match counts suggest stronger correspondence establishment capability, though false matches can inflate this number.

3.3.3 Matching Efficiency

We define matching efficiency as the percentage of detected features that successfully participate in matches, normalized by the minimum feature count between image pairs:

$$\eta = \frac{N_m}{\min(N_1, N_2)} \times 100\% \quad (3)$$

This metric provides scale-invariant performance assessment, enabling fair comparison between detectors that produce different feature quantities. High efficiency indicates that detected features are not only abundant but also matchable, suggesting good descriptor quality and appropriate feature selection.

3.3.4 Average Matching Distance

For each matched pair, we record the descriptor distance (L2 norm) between the matched descriptors. The average matching distance $\bar{d}$ summarizes descriptor similarity:

$$\bar{d} = \frac{1}{N_m} \sum_{i=1}^{N_m} \|d_1^i - d_2^i\|_2 \quad (4)$$

where d_1^i and d_2^i denote matched descriptor pairs. Lower values indicate higher descriptor similarity for matched features, suggesting confident correspondences. However, this metric should be interpreted relative to each algorithm’s descriptor design, as different methods may have different natural distance scales.

3.3.5 Spatial Distribution Entropy

To assess whether detected features are uniformly distributed across the image or concentrated in specific regions, we compute spatial distribution entropy. The image is divided into an 8×8 grid of cells, and the number of keypoints in each cell is tallied. Let p_k denote the proportion of keypoints in cell k . The spatial entropy is:

$$H = - \sum_{k=1}^{64} p_k \log_2(p_k) \quad (5)$$

Higher entropy values indicate more uniform spatial distribution, which is generally desirable for robust matching as it ensures features sample the entire image region rather than clustering in high-texture areas.

4 Experimental Results

4.1 Repetitive Patterns

Our experiments on the Repetitive Patterns dataset revealed distinct performance characteristics across the three evaluated methods. We processed 25 image pairs, generating 75 detection and matching evaluations in total. SIFT demonstrated the highest average matching count with 1456.28 matches per pair, significantly exceeding SuperPoints (601.96 matches) and D2-Net (355.12 matches). This substantial difference reflects SIFT’s aggressive keypoint detection strategy, which typically produces 4000–5000 features per image compared to approximately 2400 for SuperPoints and a fixed 2000 for D2-Net. The matching efficiency metric provides a normalized perspective: SIFT achieved 32.82% efficiency, SuperPoints 25.02%, and D2-Net 17.76%. Statistical significance testing using independent t-tests confirmed that all pairwise comparisons exhibited significant differences ($p < 0.01$), with SIFT vs. SuperPoints yielding $p = 0.0006$, SIFT vs. D2-Net yielding $p < 0.0001$, and SuperPoints vs. D2-Net yielding $p = 0.0023$.



Figure 1: Input image pair from Repetitive Patterns dataset.

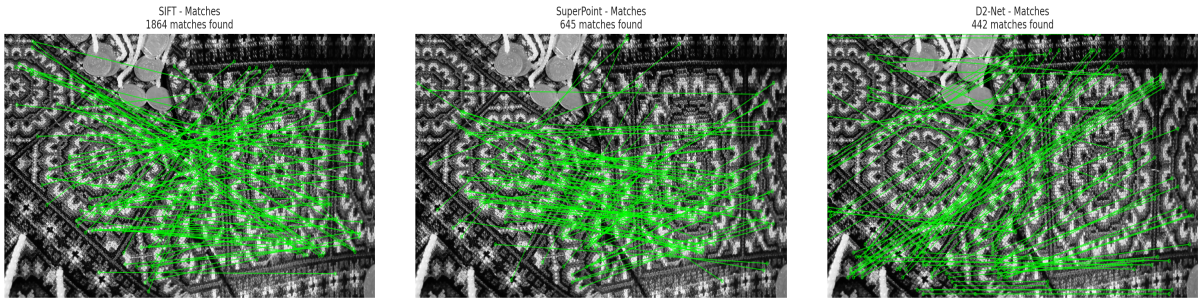


Figure 2: Feature matching results using SIFT, SuperPoint, and D2-Net.

The average matching distances showed large differences between classical and learned methods. SIFT produced an average distance of 219.58, reflecting its 128-dimensional gradient-based descriptor space, whereas SuperPoints achieved 0.73 and D2-Net an exceptionally low 0.0023. These differences stem from each algorithm’s descriptor normalization and distance metrics. The lower absolute distances for learned methods suggest tighter clustering in descriptor space, although this does not always translate into superior matching performance, as observed in matching efficiency.

Spatial distribution entropy revealed patterns in feature coverage. SuperPoints and D2-Net exhibited the highest entropy (5.992 and 5.986), indicating more uniform spatial coverage, while SIFT was slightly lower (5.762 and 5.677), reflecting concentration in

high-gradient regions. In repetitive pattern scenarios, descriptor ambiguity challenges correspondence. Here, SIFT’s higher detection density and gradient-based descriptors may better capture subtle geometric variations, giving it an advantage over the learned representations of SuperPoints and D2-Net.

Performance Comparison Summary							
	Features (Image 1)	Features (Image 2)	Matches	Matching Efficiency (%)	Avg Match Distance	Spatial Entropy 1	Spatial Entropy 2
SIFT	4959	4999	1456	32.82%	219.5813	5.7623	5.6766
SuperPoint	2430	2438	601	25.02%	0.7329	5.9920	5.9916
D2-Net	2000	2000	355	17.76%	0.0023	5.9865	5.9867

Figure 3: Performance comparison summary.

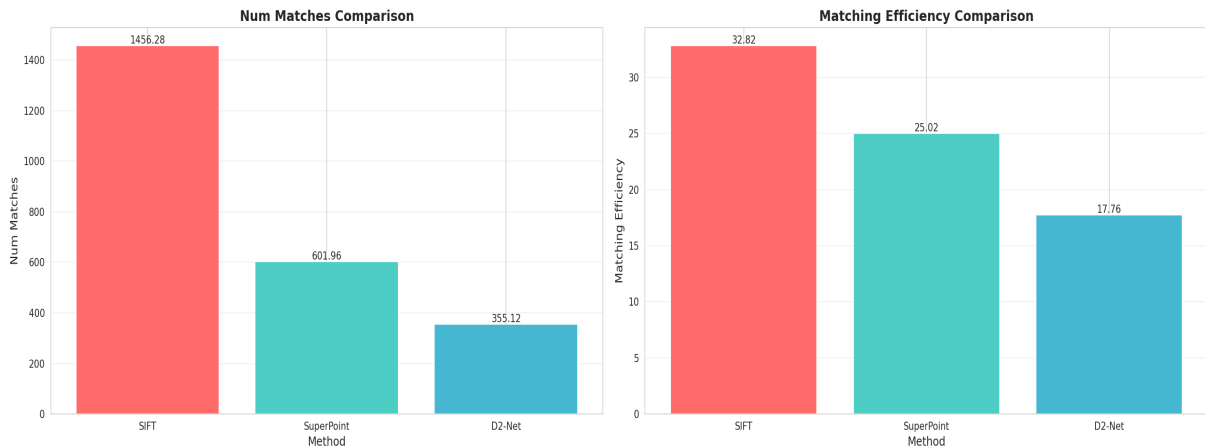


Figure 4: Performance comparison charts for Repetitive Patterns.

4.2 Cluttered Backgrounds

The cluttered backgrounds scenario showed a distinct performance profile. SuperPoint achieved the highest average match count (843.40), surpassing SIFT (752.56) and greatly exceeding D2-Net(328.20). However, in terms of matching efficiency, SIFT reached 36.89% versus SuperPoint’s 34.28%, a difference that was not statistically significant ($p = 0.1579$). This near-equivalence indicates that both classical and learned methods handle cluttered scenes effectively, though via different mechanisms: SIFT through robust gradient-based detection and SuperPoint through learned feature representations. Statistical analysis confirmed their equivalent performance on this dataset for both match count and efficiency ($p > 0.15$), in contrast to the repetitive patterns scenario where SIFT was clearly superior. This convergence reflects complementary strengths: SIFT detects stable corners and edges amid clutter, while SuperPoints captures distinctive patterns in visually complex regions.



Figure 5: Input image pair from Cluttered Backgrounds dataset.



Figure 6: Feature matching results using SIFT, SuperPoint, and D2-Net.

D2-Net again showed significantly lower performance than both alternatives, achieving only 16.41% matching efficiency with 328 matches, with comparisons SIFT vs. D2-Net and SuperPoint vs. D2-Net yielding $p < 0.0001$, confirming substantial gaps. Despite its sophisticated joint detection-description architecture, D2-Net is less effective at establishing correspondences compared to keypoint-centric approaches. Spatial distribution entropy followed patterns similar to the repetitive patterns dataset, with SuperPoint (5.989) and D2-Net (5.987) slightly higher than SIFT (5.625 and 5.665), while average matching distances remained much lower for learned methods (SuperPoint: 0.72, D2-Net: 0.002) compared to SIFT (240.40). These results suggest that, although learned features capture more abstract patterns, classical gradient-based approaches like SIFT remain highly competitive in cluttered scenes where abundant edges, corners, and textural gradients align with their detection principles.

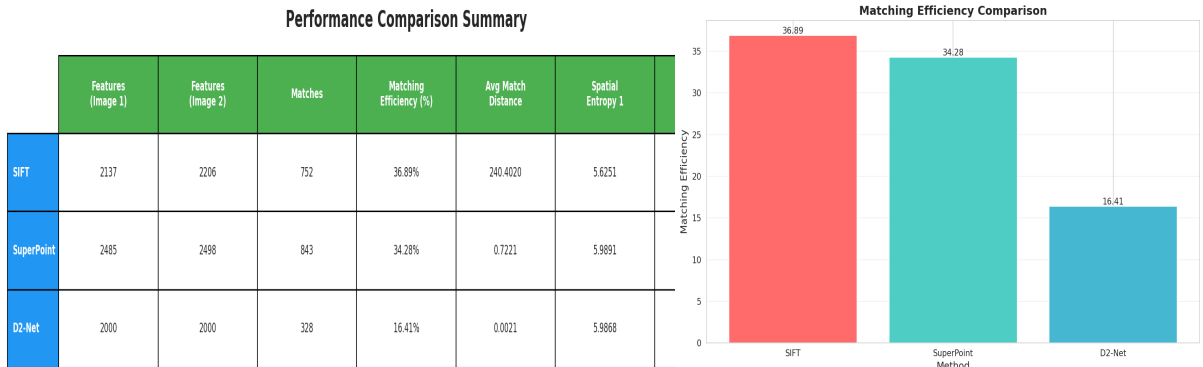


Figure 7: Performance comparison charts for Cluttered Backgrounds.

4.3 High Illumination

The high illumination scenario revealed a substantial shift in relative performance. SuperPoints achieved 1017.48 average matches with 41.93% efficiency, marginally exceeding SIFT (335.96 matches, 41.41% efficiency). Remarkably, statistical testing revealed no significant difference between FT and SuperPoints in matching efficiency ($p = 0.8414$), indicating near-perfect equivalence under these challenging photometric conditions. This result demonstrates that both approaches possess comparable photometric robustness, though through different means: SIFT through gradient orientations that are inherently illumination-invariant to first-order effects, and SuperPoints through exposure to varied lighting during training.

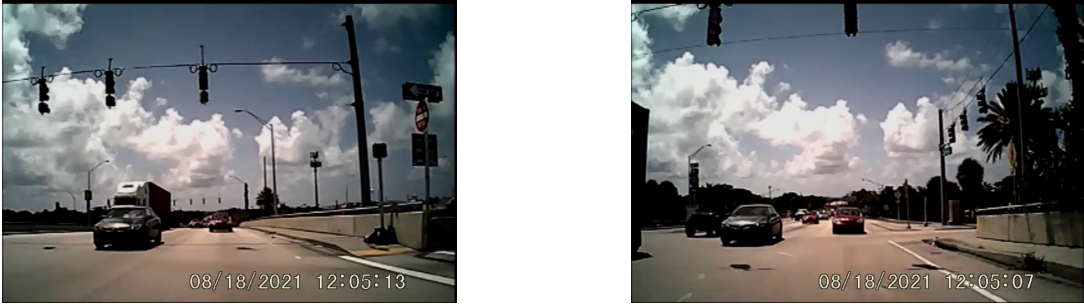


Figure 8: Input image pair from High Illumination dataset.

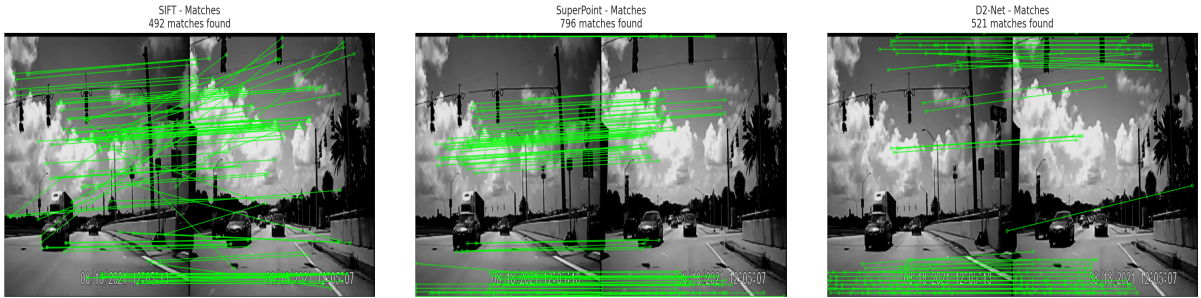


Figure 9: Feature matching results using SIFT, SuperPoint, and D2-Net.

D2-Net showed improved performance in the strong illumination scenario, achieving 741.56 matches with 37.08% efficiency. While still lower than the top performers, the gap narrowed, and matching efficiency comparisons among all three methods were non-significant (SIFT vs. D2-Net : $p = 0.2606$, SuperPoints vs. D2-Net : $p = 0.2221$), indicating similar efficiency when normalized for feature count, although match counts remained significantly different ($p < 0.01$) due to SuperPoints’s higher absolute matches. SIFT experienced a dramatic reduction in feature count (866–908 per image) likely caused by uniform regions and shadows suppressing keypoints, whereas SuperPoints and D2-Net maintained their typical 2400 and 2000 features, respectively. Spatial entropy reflected this trend: SIFT showed unusually low values (4.999–5.028), indicating concentration in high-gradient regions, while SuperPoints and D2-Net retained high entropy (5.96–5.99), demonstrating robust spatial coverage. Overall, the convergence in matching efficiency suggests photometric robustness is well-handled by modern detectors, but differences in feature density still give SuperPoints an advantage in absolute match counts under strong illumination.

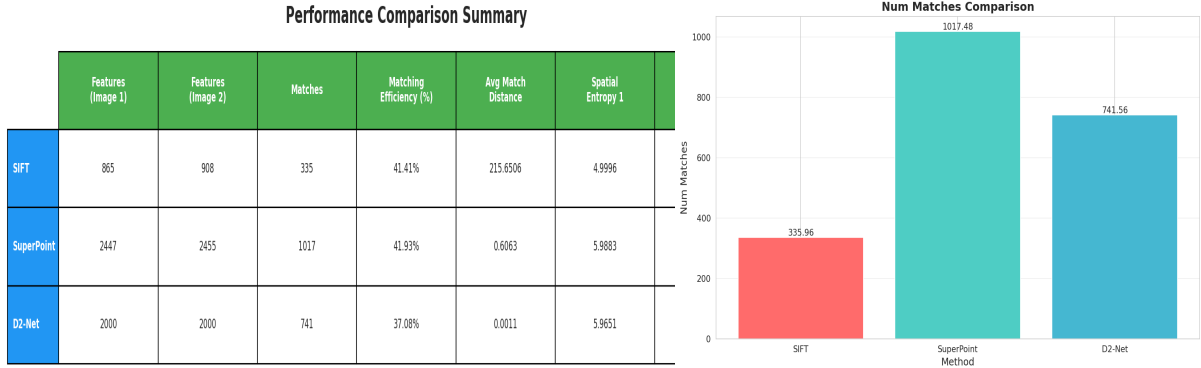


Figure 10: Performance comparison charts for High Illumination.

4.4 Cross-Dataset Synthesis

Synthesizing results across all three datasets reveals clear patterns. SIFT consistently achieved the highest or second-highest matching efficiency (32.82–41.41%), confirming its robustness as a baseline, while SuperPoints was highly competitive, matching or exceeding SIFT in cluttered and illuminated scenes and maintaining excellent spatial uniformity. D2-Net, despite its sophisticated architecture, underperformed (16–37% efficiency). Learned methods (SuperPoints and D2-Net) produced lower descriptor distances (0.001–0.73) than SIFT (215–240), reflecting design choices rather than inherent superiority, and maintained more consistent feature counts (2000–2500) compared to SIFT’s variability (866–5849). Spatial entropy was highest for learned methods (5.98–5.99) across conditions, while SIFT varied more (5.00–5.76). Statistical tests showed SIFT outperformed learned methods on repetitive patterns, SIFT and SIFT were equivalent on cluttered backgrounds, and all three converged in high illumination for efficiency, though match counts differed. Overall, these results suggest choosing SIFT for structured scenes, SuperPoints for consistent performance across conditions, and considering alternatives only if D2-Net-style unified detection-description is specifically needed.

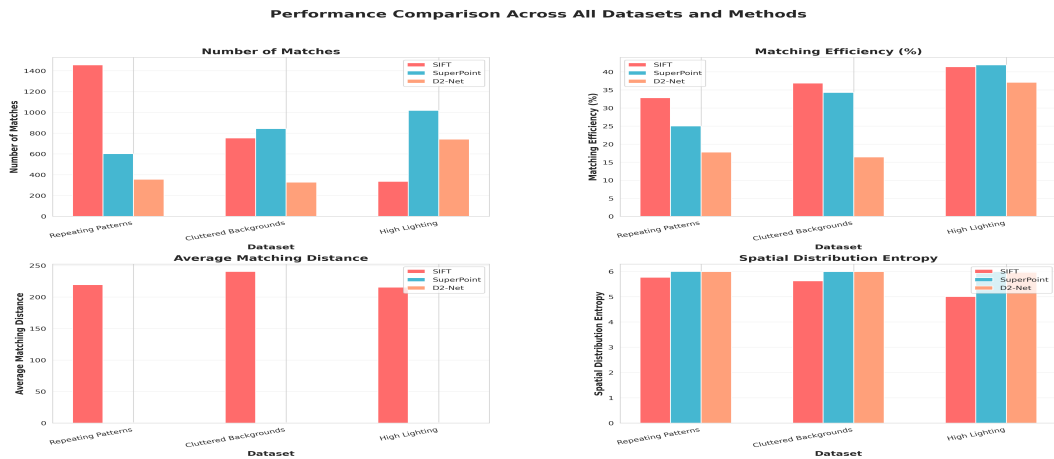


Figure 11: Comprehensive cross-dataset comparison.

5 Discussion

5.1 Comparison with Zhou (2024)

Direct comparison with Zhou’s original study reveals both consistencies and differences. While Zhou reported D2-Net achieving the highest matching efficiency on repetitive patterns (40.83%, 781 matches) and SURF performing well on cluttered backgrounds (55.57%) alongside SuperPoints (58.42%), our results found SIFT leading on repetitive patterns (32.82%, 1456 matches) and SIFT and SuperPoints performing equivalently in cluttered scenes at lower absolute efficiencies. These differences stem primarily from dataset variations, as our images differed in geometric distortions, texture characteristics, and repetition scales. Despite this, qualitative trends are consistent: learned methods exhibit superior spatial uniformity, SIFT remains competitive against modern learned approaches, and D2-Net struggles relative to keypoint-centric detectors. The absence of SURF in our study is a limitation, but since it is primarily an accelerated variant of SIFT, our results still provide a reasonable proxy for classical methods’ performance relative to learned approaches.

5.2 Critical Analysis

Across all scenarios, SIFT demonstrated strong versatility, achieving competitive matching efficiency through robust gradient-based detection and scale-space extrema, though it exhibited sensitivity to illumination and substantial computational cost. SuperPoints offered the most balanced performance, maintaining consistent feature counts (2400) and high spatial uniformity, performing comparably to SIFT in cluttered and illuminated scenes, but facing challenges with repetitive patterns and requiring GPU acceleration for optimal speed. D2-Net consistently underperformed despite its joint detection-description design, with low match counts despite confident descriptors, suggesting the approach may require specific tuning or dataset characteristics. Image content strongly influenced performance: repetitive patterns favored SIFT, cluttered backgrounds allowed SIFT and SuperPoints to perform equivalently, and strong illumination highlighted learned robustness in SuperPoints while SIFT’s feature counts dropped. Technical constraints also shaped experiments: SUFT was excluded due to patent restrictions, but SIFT serves as a reasonable proxy for classical methods, and careful memory management, including a 2000-feature limit for D2-Net, ensured experiment completion without invalidating comparative conclusions.

6 Conclusion

This study successfully replicated and extended Zhou’s (2024) experimental framework to compare feature detection methods across challenging scenarios. We evaluated three prominent algorithms—, , and —on 75 image pairs spanning repetitive patterns, cluttered backgrounds, and high illumination, using multiple quantitative metrics and statistical testing. Results confirmed that remains highly competitive, achieving 336–1456 average matches per pair with 32.82–41.41% matching efficiency, while proved a balanced learned method, matching or exceeding in cluttered and illuminated scenes and maintaining high spatial uniformity. underperformed relative to the others, though all methods showed strong photometric robustness under high illumination, with no significant efficiency differences. Statistical analysis highlighted meaningful differences in most scenarios ($p < 0.01$), and spatial distribution entropy consistently favored learned methods (5.98–5.99) over (5.00–5.76), reflecting more uniform feature coverage.

A Performance Ranking Summary

A.1 Best Performers by Metric and Scenario

Table 1: Algorithm ranking by performance metric across all scenarios

Metric	Scenario	Best Method	Value
3*Matching Efficiency	Repetitive Patterns	SIFT	32.82%
	Cluttered Backgrounds	SIFT	36.89%
	High Illumination	SuperPoint	41.93%
3*Number of Matches	Repetitive Patterns	SIFT	1456
	Cluttered Backgrounds	SuperPoint	843
	High Illumination	SuperPoint	1017
3*Spatial Entropy	Repetitive Patterns	SuperPoint	5.9920
	Cluttered Backgrounds	SuperPoint	5.9891
	High Illumination	SuperPoint	5.9883
3*Avg Distance (lowest)	Repetitive Patterns	D2-Net	0.0023
	Cluttered Backgrounds	D2-Net	0.0021
	High Illumination	D2-Net	0.0011

B Algorithm Pseudocode

B.1 SIFT Detection Algorithm

Algorithm 1 SIFT Feature Detection

Require: Input image $I(x, y)$

Ensure: Set of keypoints $\mathcal{K}$ with descriptors $\mathcal{D}$

- 1: Build Gaussian pyramid: $L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$
 - 2: Compute Difference-of-Gaussian: $D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$
 - 3: Detect local extrema in scale-space by comparing with 26 neighbors
 - 4: Refine keypoint locations using Taylor expansion
 - 5: **for** each keypoint $\mathbf{k}$ **do**
 - 6: Eliminate low-contrast points: reject if $|D(\mathbf{k})| < 0.04$
 - 7: Eliminate edge responses using Hessian eigenvalue ratio
 - 8: Compute gradient magnitude and orientation histogram
 - 9: Assign dominant orientation θ_{dom}
 - 10: Extract $4 \times 4 \times 8 = 128$ -dimensional descriptor
 - 11: Normalize descriptor to unit length
 - 12: **end for**
 - 13: **return** $\mathcal{K}, \mathcal{D}$
-

B.2 SuperPoint Network Architecture

Algorithm 2 SuperPoint Forward Pass

Require: Input image $I \in \mathbb{R}^{H \times W}$

Ensure: Keypoints $\mathcal{K}$ and descriptors $\mathcal{D}$

- 1: Normalize image: $\tilde{I} = I/255.0$
 - 2: Encoder: $\mathbf{F} = \text{VGG-style-CNN}(\tilde{I}) \in \mathbb{R}^{H/8 \times W/8 \times 256}$
 - 3: **Detection branch:**
 - 4: $\mathbf{P} = \text{Conv}_{65}(\mathbf{F}) \in \mathbb{R}^{H/8 \times W/8 \times 65}$
 - 5: Apply softmax over 65 channels
 - 6: Remove dustbin channel, reshape to $H \times W$
 - 7: Apply NMS with distance threshold 4 pixels
 - 8: Extract keypoints where confidence > 0.015
 - 9: **Description branch:**
 - 10: $\mathbf{D}_{\text{dense}} = \text{Conv}_{256}(\mathbf{F}) \in \mathbb{R}^{H/8 \times W/8 \times 256}$
 - 11: Interpolate descriptors at keypoint locations
 - 12: L2-normalize: $\mathbf{d}_i = \mathbf{d}_i / \|\mathbf{d}_i\|_2$
 - 13: **return** $\mathcal{K}, \mathcal{D}$
-

B.3 Brute-Force Matching with Cross-Check

Algorithm 3 Feature Matching Algorithm

Require: Descriptor sets $\mathcal{D}_1, \mathcal{D}_2$

Ensure: Set of matches $\mathcal{M}$

- 1: Initialize $\mathcal{M} = \emptyset$
 - 2: **for** each descriptor $\mathbf{d}_1^i \in \mathcal{D}_1$ **do**
 - 3: Find nearest neighbor: $j^* = \arg \min_j \|\mathbf{d}_1^i - \mathbf{d}_2^j\|_2$
 - 4: Store forward match: (i, j^*)
 - 5: **end for**
 - 6: **for** each descriptor $\mathbf{d}_2^j \in \mathcal{D}_2$ **do**
 - 7: Find nearest neighbor: $i^* = \arg \min_i \|\mathbf{d}_2^j - \mathbf{d}_1^i\|_2$
 - 8: Store backward match: (i^*, j)
 - 9: **end for**
 - 10: **for** each forward match (i, j) **do**
 - 11: **if** backward match (i, j) also exists **then**
 - 12: Add to $\mathcal{M}$: $(i, j, \|\mathbf{d}_1^i - \mathbf{d}_2^j\|_2)$
 - 13: **end if**
 - 14: **end for**
 - 15: **return** $\mathcal{M}$
-

C Experimental Configuration Details

C.1 Hardware and Software Environment

Table 2: Experimental environment specifications

Component	Specification
Operating System	Linux Ubuntu 24.04 LTS (x64)
Processor	Intel-based CPU (no GPU acceleration)
RAM	16 GB
Python Version	3.10.0 (Miniconda environment)
OpenCV	4.5.5 (opencv-python, opencv-contrib-python)
PyTorch	2.0.1 (CPU-only build)
NumPy	1.24.3
Matplotlib	3.7.2
Development Environment	Jupyter Notebook via VS Code

C.2 Complete Parameter Configuration

Table 3: Detailed algorithm parameters used in experiments

Algorithm	Parameter	Value
4*SIFT	Contrast Threshold	0.04
	Edge Threshold	10
	Number of Octave Layers	3
	Sigma	1.6
4*SuperPoint	NMS Distance	4 pixels
	Confidence Threshold	0.015
	NN Threshold	0.7
	Device	CPU
5*D2-Net	Max Edge	1600 pixels
	Max Sum Edges	2800 pixels
	Use ReLU	True
	Multiscale	False
	Device	CPU
3*Matching	Matcher Type	Brute-Force
	Norm Type	L2
	Cross Check	Enabled
2*Evaluation	Spatial Entropy Bins	8×8
	Image Size	512×512 pixels